

Diamonds - Analysis

Problem: Cheng Zhong, Analysis: Iliyan Yordanov

We are given a directed weighted graph. The condition that each vertex has at least one outgoing edge means there are no dead ends – there is always an edge to continue along. Unlike classic graph problems, which maximize the total sum of weights, here the comparison is based on lexicographic order. We have to find the sum of the first K edges on a lexicographically largest path (comparing the weights).

At first glance, this might seem easier: a straightforward simulation appears sufficient; we begin the traversal at the starting vertex of the edge with the largest weight, and in each round, we choose the currently accessible edge with the largest weight. However, the situation is more complex because multiple edges may have the same weight, leading to frequent ties for the maximum. Therefore, if we rely solely on simulation, each decision may involve multiple tied maximums, resulting in exponential time complexity.



Subtask 1

The small constraints allow a direct BFS-like approach for the solution. We perform a simulation where round 0 begins with all vertices active, and in each round, we select the currently accessible edges with the largest weight and store their endpoints for the next round. The answer is obtained by storing the sum of the weights for each round.

Code: `diamonds6.cpp`

Time complexity: $O(N + M^K)$.

Memory complexity: $O(N + M^K)$.



Subtask 2

It is easy to see that the exponential behavior of the previous approach is because in each round we unnecessarily repeat the same vertices multiple times. We can use a boolean array to flag the vertices that have already been added to the next round or alternatively store each round's vertices in a boolean array (instead of an integer vector).

Code: `diamonds5.cpp`

Time complexity: $O(K(N + M))$.

Memory complexity: $O(M + KN)$ or $O(M + N)$ if we only store previous and current round.



Subtask 3

This subtask is a major step toward the full solution because K is large. We can see that there is an optimal construction for the lexicographically largest path, since after sufficiently many rounds, some edges (and vertices) must repeat because they belong to the lexicographically largest sequence of weights among all possibilities.

Lemma 1: There exists a lexicographically largest path that begins with a simple path (without duplicating vertices) and then repeats a simple cycle.

[go to proof](#)

If we know the optimal construction, it becomes a straightforward task to compute the answer. The sequence of weights starts with some numbers and then it becomes periodic. For greater K , we have to find the sum of the terms of a periodic sequence.

There are two approaches for the task – one is by doing smarter simulation round by round and the other is by computing a dp to find the optimal decisions. Both approaches require a bound on the number of iterations. One can experimentally or intuitively find that this number of iterations is linear. In fact, two paths have identical sequences of weights for all rounds if and only if they have identical sequences of weights for the first $2N$ rounds. We will discuss this in detail in the next subtask.

The first approach is a continuation of the idea in the previous subtasks. Let's consider we have made sufficiently many rounds (at least $2N$). This means the non-optimal paths considered initially have already been eliminated, and we are now repeating the cycles of the possible optimal constructions. However, it is not straightforward, since the optimal paths can still be continued non-optimally for a couple of rounds (and then eliminated), after that these non-optimal continuations reappear, and so on. We can see that the only certain thing is that the starting vertices of all remaining paths need to be part of an optimal construction, otherwise they would have been eliminated already. So for each (r, v) , where r is the round and v is the vertex, it is enough to store a $(r - 1, u)$ it came from (if there are multiple options, it is enough to store one). Then, after $2N$ rounds, we pick an arbitrary vertex, recover the path, and examine the starting part, which must be an optimal construction (a path and then a cycle).

The other approach is to use dynamic programming $dp[r][v]$ = the optimal sequence of weights starting from vertex v and having r rounds (which is also the length of the sequence). Let $w(v, u)$ denote the weight of edge (v, u) . We have the following recursive relation:

$$dp[r][v] = \max_{u: \text{for all edges } (v, u)} \text{lexicographically } \{w(v, u), dp[r - 1][u]\}.$$

As the constraints are smaller, we can compute the dp for sufficiently many rounds R

(at least $2N$) fast enough without optimizations. We pick the optimal sequence from $dp[R][0], dp[R][1], \dots, dp[R][N - 1]$. There are different ways how exactly we can find the answer. We can store additional information so we can find the vertices of the edges that form the optimal sequence and then find the optimal construction from the vertices.

Another way, used in the implementations, is to use only the sequence of weights. For this R needs to be at least $3N$. We know the first elements must come from the path of an optimal construction, so to identify the repeating cycle we temporarily ignore the first N elements. As the length of the repeating cycle in the optimal construction is at most N , we can find any valid repeating cycle in the sequence of the last $2N$ elements.

The only drawback with the second approach is the memory if we store directly the sequences in each state. We have to be more efficient here and store the sequences as linked lists so $dp[r][v]$ is only the head element of the sequence, and that element is linked to some optimal (for v) $dp[r - 1][u]$.

	First approach	Second approach
Code	<code>diamonds_iliyan_n2_mem.cpp</code>	<code>diamonds4.cpp</code>
Time complexity	$O(NM)$	$O(N^2M)$
Memory complexity	$O(M + N^2)$	$O(M + N^2)$



Subtask 4

The constraints for this subtask make the graph consist of a single cycle or multiple disconnected cycles. This subtask is given so that competitors can think more about the cycles and the needed rounds for comparison. We can find the optimal traversal by comparisons:

- For two traversals in the same cycle, we only need to compare their results of the first $length(cycle)$ rounds;
- For two traversals in different cycles, we only need to compare their results of the first $lcm(length(cycle_1), length(cycle_2))$ rounds.

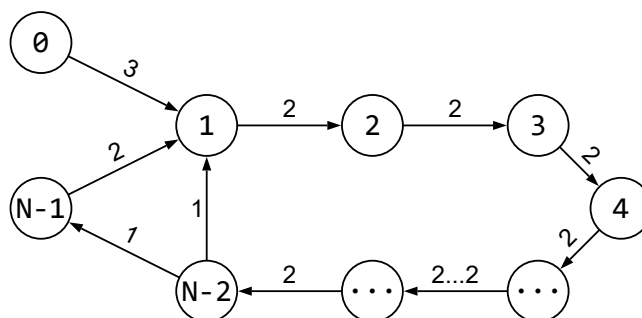
Although not necessary for this subtask, we can use the following lemma, which shows that only a linear number of comparisons are needed for the second case.

Lemma 2: If two periodic sequences with periods p and q have the same first $p + q$ terms, then they are identical.

[go to proof](#)

There is an even stronger bound: it is enough to compare the first $p + q - \gcd(p, q)$ terms but this is not needed for the task and does not change the complexity. The lemma tells us that we need approximately $2N$ rounds to compare any two cycles. In the general case

when the optimal construction starts with a path this would require approximately $3N$ rounds (additional N to enter the cycle). But it is easy to see that still $2N$ rounds are enough because the cycle and the path have to be disjoint (they cannot share vertices). This is a tight bound as illustrated in the following example, where $2N - 3$ rounds are required to compare the two possible optimal constructions (there are also such tests with a modified version):



Code: `diamonds9.cpp`

Time complexity: $O(N^2)$.

Memory complexity: $O(N)$.

Subtask 5

The task becomes much simpler when all $d[i]$ are distinct. There is only one maximum choice for each round, so the only challenge is handling large K . It is easy to see that the optimal construction is first a simple path and then repeating a simple cycle. So we follow the maximum weight for each round until we find a cycle, and then we have to find the sum of the terms of a periodic sequence. This subtask allows contestants to observe the optimal construction more easily.

Code: `diamonds_iliyan_unique_d.cpp`

Time complexity: $O(N + M)$.

Memory complexity: $O(N + M)$.

Subtask 6

This subtask is similar to the previous one as the optimal construction is much easier to see than in the general case. Let edge (u, v) be the one with weight 2. We should use the weight 2 as often as possible. We have to start with (u, v) and then return as quickly as possible (using the fewest number of weight-1 edges) to vertex u . If this is not possible, the answer is simply $2 + (K - 1) = K + 1$. Otherwise we have to compute how many times we will use weight 2 and the answer is $K +$ this number. We can use BFS to find the shortest path of 1-s from v to u or determine that no such path exists.

Code: `diamonds8.cpp`

Time complexity: $O(N + M)$.

Memory complexity: $O(N + M)$.



Subtask 7

We now revisit the two approaches and discuss the different ways to optimize them to fit the small memory constraint. There are two ways to optimize the first approach, which relied on smarter round-by-round simulation. We can use a `bitset` memory optimization. Instead of storing, for each (r, v) , a $(r - 1, u)$ it came from, we can use one `bitset` per round to record the vertices that are present. If we know the vertices in consecutive rounds it is easy to find a path that starts from $(0, v)$ and goes to (R, u) for some vertices v and u , and a final round R . If we are at (i, x) and we have an edge (x, y) in the original graph, we have to check if vertex y was present in the next round $i + 1$, i.e. if $(i + 1, y)$ existed during the simulation. Once we find a path, we can compute the answer as before.

The second way to optimize the first approach uses the earlier idea of finding the answer using only the sequence of weights. We can store the maximum weight that is used for each round and construct the optimal sequence of weights without knowing the exact path. Then, as described earlier, we can find the cycle by examining the last $2N$ elements and then compute the final answer. (we still need at least $3N$ rounds of simulation)

For the second approach, we will first optimize the time complexity. We recall the recursive relation:

$$dp[r][v] = \max_{u: \text{for all edges } (v, u)} \text{lexicographically } \{w(v, u), dp[r - 1][u]\}.$$

The bottleneck is the "max lexicographically" comparison, which we previously did in linear time, but this can be improved. We will keep a sorted list of $dp[r - 1][v]$ (for fixed $r - 1$) and we need to obtain a sorted list of $dp[r][v]$. For this we can store in $dp[r - 1][v]$ "ranks" which are numbers that show the relative order of the weight sequences. To obtain the sorted list of $dp[r][v]$, for each v we use $w(v, u)$ as a primary key and $dp[r - 1][u]$ as a secondary key, where u is the argmax in the relation above for v . Then we sort these pairs lexicographically and find the new ranks for $dp[r][v]$. We can even do this in linear time as we can do a `radix sort` over the pairs. It is also possible not to fix the number of iterations and terminate when the ranks stop changing.

Now we have good time complexity and we will find the optimal construction in the following way. Suppose we have all $dp[R][v]$ for some number of rounds R , and we construct a new graph with the edges (v, u) for each v , where u is the argmax in the `dp` relation for $dp[R][v]$. We claim that if R is at least $2N$ and we start from the optimal v (with the highest rank $dp[R][v]$), and follow these edges, we will find an optimal construction. This is because all paths with such length starting from v must follow an optimal construction so even if there are ties for argmax for v , all of them will be part of optimal constructions. A similar argument holds for the other vertices we visit along the path since non-optimal possibilities would be eliminated for large enough R . Thus, we directly obtain the optimal

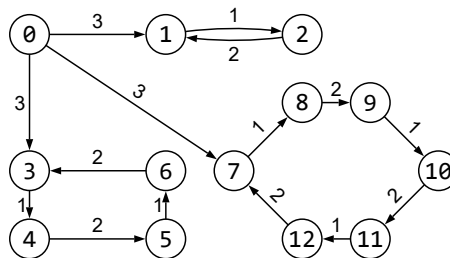
construction, from which the answer follows as before. For this solution, $R = N$ is also sufficient, since the dp takes a more global view and removes non-optimal possibilities from the beginning, and needs additional iterations to propagate that information.

	First approach	Second approach
Code	diamonds_encho_bitset.cpp, diamonds7.cpp	diamonds1.cpp, diamonds_iliyan.cpp
Time complexity	$O(NM)$	$O(NM + N^2 \log N)$ or $O(NM + N^2)$
Memory complexity	$O\left(M + \frac{N^2}{64}\right)$ for the bitset and $O(N + M)$ for the other solutions	$O(N + M)$



Final thoughts

During testing, we found a particular variation of the first approach. Instead of reasoning about the number of rounds, we can terminate if we repeat the same set of vertices during a round (which we can find using hashing). Then the cycle is the sequence of weights between the two repetitions. It turns out exponentially many rounds may be required for the repetition to happen so we still require a bound for the rounds. The best type of test we came up with to battle such cheat solutions is the following:



The base model is we have some constant k and then we make cycles of lengths $k, 2k, 3k, 5k, 7k, 11k, \dots$ that repeat $1, 2, \dots, k$ as weights (in the example $k = 2$ and the cycles are of lengths 2, 4 and 6). In this way we need the LCM of the lengths (a huge value), as the number of rounds required to repeat the same set of vertices. Also if we terminate early, it is not simple to compute the answer looking at the sequence of weights, since we still have a non-trivial repeating cycle (the simplest correct way is to have a full solution like `diamonds7.cpp`). Unfortunately, it turned out we had missed adding these tests in the official test data so a few contestants passed by terminating early and computing the answer in a wrong way.

The original proposal for this task was even harder, but we believe the current variant is more appropriate for EJOI. In fact, it turned out to be slightly harder for contestants than we initially thought.



Proof of Lemma 1

Lemma 1: There exists a lexicographically largest path that begins with a simple path and then repeats a simple cycle.

Suppose a lexicographically largest path is: $L := (v_0, v_1), (v_1, v_2), \dots$. As the path is infinite and the different vertices are only N , there has to be a repeated vertex. Let the first repeated vertex be v_j , with its previous occurrence at v_i ($0 \leq i < j$). This also means that $v_0, v_1, \dots, v_{i-1}$ is a simple path and $v_i, v_{i+1}, \dots, v_{j-1}, v_i$ ($v_i = v_j$) is a simple cycle. Let $P := (v_0, v_1), (v_1, v_2), \dots, (v_{i-1}, v_i)$ and $C := (v_i, v_{i+1}), (v_{i+1}, v_{i+2}), \dots, (v_{j-2}, v_{j-1}), (v_{j-1}, v_i)$.

We will prove the following claim by induction: for each $r \geq 1$, the sequence of weights in L starts with the sequence of weights of $\{P, C^{(r)}\}$, where $C^{(r)}$ denotes $\underbrace{C, \dots, C}_r$.

Base case $r = 1$. We know that the path L starts with $\{P, C\}$ so the claim holds.

Induction step. Let the claim be true for some $r \geq 1$. We will prove it for $r + 1$.

The claim is true for r , so the weights of L start with the weights of $\{P, C^{(r)}\}$, and we will denote the edges after that with S . We will also denote $P' := \{C, S\}$.

The sequence of weights of L equals that of $\{P, C^{(r-1)}, P'\}$. Since $\{P, C^{(r-1)}, S\}$ is a valid path (with one fewer cycle), P' must be lexicographically larger than or equal to S . If they are equal, then we are ready as this would mean that the weights of S are starting with the weights of C (since $P' = \{C, S\}$), so P' is starting with weights $C^{(2)}$ and L is starting with weights $\{P, C^{(r+1)}\}$.

Let's assume they are not equal. So there must be an edge in P' with larger weight than the corresponding one in S . Let the first such edge be e . If e is in the cycle C , then the cycle will be lexicographically larger than the corresponding part of S , so a path $\{P, C^{(r+1)}\}$ will be lexicographically larger than L , which is a contradiction.

The only remaining case is when e is after the cycle C . This would mean that the sequence of weights in the cycle C is the same as the corresponding sequence of weights in S , which means that the weights of S are starting with the weights of C , and L is starting with weights $\{P, C^{(r+1)}\}$.

Thus, the claim follows by induction. This means that the sequence of weights in L is the same as the sequence of weights of $\{P, C, C, \dots\}$. Since $\{P, C, C, \dots\}$ is a valid path, it is a lexicographically largest path that begins with a simple path and then repeats a simple cycle. $\square$



Proof of Lemma 2

Lemma 2: If two periodic sequences with periods p and q have the same first $p + q$ terms, then they are identical.

Without loss of generality, assume $p \leq q$. Let the sequences be $P_0, P_1, \dots$ and $Q_0, Q_1, \dots$

We will prove the following claim by induction: for each $r \geq 0$, the sequences have the same first $p + q + rp$ terms.

Base case $r = 0$. As the sequences have the same first $p + q$ terms, the claim holds.

Induction step. Let the claim be true for some $r \geq 0$. We will prove it for $r + 1$.

Let's assume the contrary and let $p + q + rp \leq i < p + q + (r + 1)p$ be an index such that $P_i \neq Q_i$. We have the following:

- As P has a period p , we have $P_i = P_{i-p}$.
- As $i - p < p + q + rp$, we have $P_{i-p} = Q_{i-p}$.
- As Q has a period q , we have $Q_{i-p} = Q_{i-p-q}$.
- So, $P_i = P_{i-p} = Q_{i-p} = Q_{i-p-q}$.

Similarly, $Q_i = Q_{i-q} = P_{i-q} = P_{i-p-q}$ (since $p \leq q$, we have $i - q \leq i - p < p + q + rp$). But $0 \leq i - p - q < p + q + rp$, so $P_{i-p-q} = Q_{i-p-q}$ by the induction step and this means $P_i = Q_i$, which is a contradiction.

Thus, the claim follows by induction. This means that the sequences must be identical, since we can pick an arbitrarily large r . □

Grid - Analysis

Idea: AmirReza PourAkhavan, Mohammad Hossein Paydar

Implementation and Analysis: Ivan Lupov

The analysis will focus on the subtasks with $N = 1$. For their two dimensional counterparts it is enough to store and process the relevant information for all columns the same way it is stored for a singular row. This way we will take care of jumps that move in the same column. Also note that for clarity we skip the row index in any mentions of a - it is implied to be equal to 0.



Subtask 1: $N = 1, M \leq 200$

The dynamic programming state $dp[x] = \text{max cost to reach position } x$ should be obvious to contestants. Here it is enough to iterate over the position $y < x$, such that $y \rightarrow x$ is the last jump before stepping on position x .



Subtask 2: $N = 1, A_j \leq A_j + 1$

The additional restriction here allows for a greedy idea: if possible, it is always better to jump as far as possible - the penalty stays the same but the difference in $|a_j - a'_j|$ can only grow. Thus it is optimal to jump from $(0, 0)$ to $(0, m - 1)$ immediately. It is worth noting that in the 2d case there appear to be two paths $(0, 0) \rightarrow (0, m - 1) \rightarrow (n - 1, m - 1)$ and $(0, 0) \rightarrow (n - 1, 0) \rightarrow (n - 1, m - 1)$ however both of them give the same number of coins.



Subtask 3: $N = 1, C = 0$

Here there is no penalty for making jumps and since $|x - y| \geq 0$, we can feel free to spam them. Making short jumps between adjacent cells in the grid is optimal and since $N = 1$ we can calculate the profit of this strategy with a simple for-loop. In the 2d case we will have to use dynamic programming.



Subtask 4: $N = 1, M \leq 50\,000$

A common idea in problems involving some cost function with absolute values (e.g: $|x - y|$) is to split its computation in two cases, depending on the sign of the inner value (here: $x - y$). Returning to our dynamic programming idea from subtask 1, here this reduces to:

$$dp[x] = \max\{\max_{y < x}\{dp[y] + a_y \mid a_x \leq a_y\} - a_x, \max_{y < x}\{dp[y] - a_y \mid a_x > a_y\} + a_x\}$$

In short, for lower (than a_x) values we care about the maximum $dp - a$ value and for



higher we care about $dp + a$ value. Here the time complexity is $O(m \log A)$ where A is the maximum value in the grid - still not optimal.



Subtask 5: $N = 1$

Here we will develop the key idea of the solution. Coding the 100-points solution is simply a matter of taking care of the columns the same way we will take care of the singular row now.

Key observation about the behaviour of the “absolute value” function is that $|x - y| = \max\{x - y, y - x\}$. This is really convenient because writing our dynamic programming transition was some equation of $|\cdot|$ and maxs, but now we can make it simply an equation of maxs.

$$\begin{aligned} dp[x] &= \max_{y < x} \{dp[y] + |a_x - a_y|\} \\ &= \max_{y < x} \{dp[y] + a_x - a_y, dp[y] + a_y - a_x\} \\ &= \max\{\max_{y < x} \{dp[y] - a_y\} + a_x, \max_{y < x} \{dp[y] + a_y\} - a_x\} \end{aligned}$$

Thus it is enough to keep track of the maximum value only for the $dp - a$ and $dp + a$ expressions. Note that we group terms such that all mentions of y can be taken care of together - this is common when rearranging equations to get a neater result.

Prison – Analysis

Idea: Sultan Alaiban and Viktor Kozhuharov, Implementation and Analysis: Emil Indzhev

1 Introduction

We can think of the problem in several ways. One of them is the one from the statement – i.e. encoding and decoding. Another is to think about generating a list of triples such that no two triples have a pair in common. We can simply generate all of these in the precompute and then just return cached values in encode. Similarly we can cache the decodings of all pairs and return using that in decode.

2 Upper bound for N

Let's start by trying to figure out an upper bound for N , as a function of M . The decoder gets an (unordered) pair and there are $M + M(M - 1)/2$ possible pairs. However, each triple maps to multiple pairs. If the triple has 3 distinct numbers, it maps to 3 possible pairs. If it has the same number 3 times, it maps to only 1 pair. It is bad to have triples with 2 equal numbers and 1 different, since those are strictly worse than the triple with the duplicate number being repeated 3 times. Therefore, the M equal number pairs can map to distinct A s and the other $M(M - 1)/2$ pairs must be in groups of 3. In total, we get $N \leq M + M(M - 1)/6$ (roughly $M^2/6$). We can see that the targets are very close to this.

3 Naive ideas

Before looking at some better solutions (ones with $N = O(M^2)$), we can briefly talk about some simpler ideas. Firstly, we can just do all triples of equal numbers ($N = M$). This gets 10.69 points.

A better idea is to also have triples of the form $(i, i + 1, i + 2)$ for all i -s that are multiples of 3. This can then be extended with $(i, i + 3, i + 6)$, for all i -s which when divided by 3 (rounded down) are multiples of 3. Then follows $(i, i + 7, i + 14)$ and so on with the offset always being double the previous offset plus 1. This ends up with $N = O(M \log M)$. This gets 23.07 points.

4 Bucket solutions

4.1 3 buckets

The problem would be a lot easier if we knew which number is which when decoding. So let's split the M numbers up into 3 buckets and the first number is always from the first bucket, the second from the second and the third from the third. That way the triple effectively becomes ordered and when decoding we know the order of the 2 numbers and also which of 3 numbers is missing.

Now, there are many ways to encode A . Easiest is to encode it using the first 2 numbers and have the third number act as a sort of checksum to recover the missing number, in case one of the first 2 is missing. An alternative, which is slightly worse, is to use the Chinese remainder theorem and have the three buckets have co-prime sizes, encoding A as its remainder modulo the 3 bucket sizes (big enough that the product of any 2 of them is $\geq N$).

Now onto the proposed, easier way: We can encode A with two numbers in base $\sqrt{N}$ using the first 2 numbers. Alternatively, we can say we just iterate through all (ordered) pairs over the first 2 numbers and each of those uniquely defines a triple. Then, as explained, we only need to figure out a scheme for the last number, so that using it and whichever of the first 2 numbers, we can recover the other one.

One easy idea is to use the XOR of the first 2 numbers. However, this needs the last bucket to be a power of 2 (note that the first 2 buckets don't need to be a full power of 2). Another idea is to just use the sum of the first 2 numbers, but this needs the last bucket to be twice as big. The simplest best thing to do is to make the last number be the sum of the first 2 modulo their bucket size (which makes all 3 buckets have the same size).

This solution generates $M + (M/3)^2$ triples (around $M^2/9$) and gets 58.96 points.

4.2 2 buckets

The issue for all of these is that we don't use any pairs where 2 numbers come from the same bucket. One improvement is to instead only have 2 buckets, the first two numbers being from the first bucket and encoding A as an unordered pair (instead of an ordered one) and the third number (from the second bucket) working the same way as before. Alternatively, we can say we iterate through all unordered pairs from the first bucket and compute the third number for each.

This solution generates $M + (M/2) * (M/2 - 1)/2$ triples (roughly $M^2/8$) and gets 67.2 points.

Regardless, the main issue remains. Here, we don't have any pairs where both numbers are from the second bucket.

4.3 Divide and conquer

We can make a clever observation which works on top of either of the above solutions. Since the only pairs we're not using are the ones where the 2 numbers are from the same bucket, they can be generated from triples where all 3 numbers are from that same bucket. In fact these are the only types of triples left available to us, since any other triples would overlap (in pairs) with already existing triples. Then, notice that generating triples where all 3 numbers are from the same bucket is the exact same problem as the full one. Thus, we just apply a standard divide and conquer approach and run our solution on the bucket (in the case of the 3 bucket solution – on each; in the case of the 2 bucket one – on the second one).

We can see that these solutions fully cover all pairs (up to rounding) and therefore get over 99 points. Find an optimized implementation of a recursive solution based on the 2 buckets one in: `emil_rec_best.cpp`

5 Explicit constructions

The above solutions are derivable if we start with a bucket based solution. However, there are also some direct constructions that work. Some would work best only for some specific types of values of M , but can be partially adapted to other ones.

In general, constructions can be produced by using any symmetric associative invertible operation and choosing the target result of applying the operation on all 3 numbers. This is because we can infer the missing number from any 2 other ones, without knowing their order (or which 2 they are).

One idea is to use XOR and use a target value of 0. Here the missing number is always the XOR of the other two numbers. We can construct the triples by iterating over all pairs and computing the third number (and then enforcing that the numbers are non-increasing/non-decreasing). Naively, this works only for M being a power of 2. A partial fix is to use numbers up to the largest power of $2 \leq M$. Better, we can use the smallest power of $2 \geq M$ and keep only the triples where all 3 numbers are less than M .

The very small issue with this is that it produces triples of the form $(X, X, 0)$, which is wasteful. We can substitute them with (X, X, X) triples, but this means we don't use 0 anymore (except for $(0, 0, 0)$). Therefore, we can pretend to add 1 to all numbers for the purposes of XORing and generate only the triples with all 3 numbers being distinct. For this solution, the best values are actually 1 less than a power of 2, but we can use the same trick (i.e. choosing a larger power and keeping only the valid triples). This solution gets 78.3 points. You can find an implementation in: `emil_constr_xor_impr.cpp`

Another idea is to use a similar to XOR operation on the base 3 representation of the numbers. The operation is per digit sum modulo 3. This being 0 means that either all 3 digits are the same or all 3 are different. These triples are actually exactly the same as the sets in the popular game Set. The approach for the solution is exactly the same as before, except we need to compute the third number digit by digit (if the first 2 number's digits match, we use the same, otherwise we use the missing digit). The (small) advantage of this operation is that that it naturally produces the triples of all 3 equal numbers. However, its best values are powers of 3, which are rarer than powers of 2. The

solution gets 59.85 points. You can find an implementation in: `emil_constr_set.cpp`

Finally, we can do a similar thing to what we did with out buckets based solutions. Just use the operation sum modulo M . Then, we take all triples which sum up to 0. Notice that we can compute the third number as the M minus the first 2 (modulo M). The small issue is again the same as with the XOR solution – we have triples of the form (X, X, Y) . There is no easy fix like with XOR, so this solution has about $N = 2M/3 + M(M-1)/6$. However, it works well on all values of M , so this is the model solution and thus gets 100 points. You can find it here: `emil_constr_modsum.cpp`

6 Greedy solution

One alternative type of solution that gets 77.91 points is a greedy one. Simply iterate over all triples of numbers in some order (say lexicographic) and, if the current triple is valid (i.e. no previous triple has any of its pairs), generate/use that triple. The issue is that this is too slow so needs to be optimized by investigating the pattern in the triples this outputs. It can be fully worked out and computed in $O(N)$ (i.e. $O(M^2)$) time (though this is fairly difficult). Find an implementation of this solution in: `encho_greedy_optimized.cpp`

Navigation – Analysis

Author: Emil Indzhev

Part I

Subtasks 1-4

1 Subtask 1: Cycle

The idea is to circle around the circle.

We need 2 colors: `INIT` and `DONE`. Always set the current node to `DONE` and go to an `INIT`. Otherwise, if no `INIT`s, terminate.

Code: `emil_cycle_2.cpp`

2 Subtask 2: Star

The idea here is that we can distinguish whether we are in a leaf or the base of the star. So we'll just visit all leaves and mark them and then terminate.

We need 2 colors: `INIT` and `DONE`. If in a leaf, set the current node to `DONE` and go to the base of the star. If in the base of the star, go to an `INIT` leaf. Otherwise, if no `INIT` leaves, terminate.

Code: `emil_star_2.cpp`

3 Subtask 3: Path

The idea is we go to 1 end of the path and then to the other and then terminate.

We need 3 colors: `INIT`, `LEFT` and `DONE`. First, if we see no `DONE`s, we are in phase 1, so we set current node to `LEFT` and move to an `INIT`. If no `INIT`s, switch to phase 2. In phase 2: set current node to `DONE` and move to an `INIT` or a `LEFT`. If no `INIT`s and no `LEFT`s, terminate. Notice that phase switching and termination can only happen in leaves.

Code: `emil_path_3.cpp`

4 Subtask 4: Tree

The idea is to run a DFS.

We need 3 colors: `INIT`, `DFS` and `DONE`. `INIT`s are unvisited nodes. `DONE`s are visited nodes whose full subtree has been explored. `DFS`s are nodes that are currently on the path from the start (root) to the current node (i.e. they are on the DFS stack).

The strategy is: if we see any `INIT`s, set the current node to `DFS` and go to an `INIT`. Otherwise, if we see any `DFS`s (there can be at most one), set the current node to `DONE` and go to the `DFS`. Otherwise (we must be in the root), terminate.

Code: `emil_tree_3.cpp`

Part II

Subtasks 5-7

5 The problem

We can try running this on a non-tree and the issue is when we need to backtrack but the current node has a back-edge. Then we see multiple DFS nodes and cannot know which way to go.

There is a “known” approach that works on general graphs, seen in some past problems (such as Robot at IOI 2023) of running a BFS using layered DFS runs that expand it by 1 layer each time. The idea is that a BFS tree has no back-edges (only edges to the previous, current or next layers). So we can record the distance from the root to the current node modulo 3 and this way we can classify each edge. Naively, such a solution might take 19 colors, but it is possible to do this with 8 colors. However, this approach cannot be used for this problem, since the number of iteration it needs is $O(N^2)$, while here we need $O(N)$ iterations.

In general, anything that is not effectively a DFS will need a superlinear number of steps. Therefore, we want to do a DFS. However, we can see that this is not possible on a general graph: In order to backtrack, we need to have a way to recover the DFS stack, which is basically a permutation. The issue is that there are $N!$ possible permutations, while there are only $N \times C^N$ possible configurations (for some constant C). Thus, we need to take advantage of the fact that the graph is a cactus.

6 Solution with $N + 1$ colors

The above argument does not hold if $C = N$. One simple solution that works even in a general graph is to run a DFS, but instead of a DFS color have N DFS-like colors, marking the depth of the node. Then when we backtrack, we just go to the deepest node with depth smaller than ours (this way we don’t even need a DONE color). If we need to backtrack from the root (which has depth 0), we terminate. This solution actually only does $2N$ iterations. It gets 15% of the points for these subtasks.

Code: `emil_cactus_bad_n+1.cpp`

7 Subtask 5

This subtask serves a few purposes.

Firstly, it allows solutions with $O(\text{MaxDegree})$ number of colors (due to storing which node is our parent), which rely on the fact that the order of `adjColors` is consistent across multiple visits to the same node. This again, works on a general graph with bounded degree.

Secondly, it simplifies some of the cases of most of the full solution ideas and allows easier elimination of some of their colors. Notably, in this subtask, each node is part of at most one cycle, which simplifies a lot of things. Additionally, since the starting node has degree 1, in most solutions we’d only start there and finish there, so we don’t need to worry about special cases due to the root.

8 Key observations

The key observation is that in a cactus back-edges are much more well behaved than ones in a general graph. This results in several concrete observations. Various solutions (and variations on these solutions) use one or both of these or further more specific observations.

1. Each node has at most 1 back-edge (this is because the parent to self edge would otherwise be in two simple cycles). Therefore, while running a DFS, the current node sees at most 2 nodes on the DFS stack.
2. If a node has a back-edge, its parent cannot have a back-edge (this is because the grandparent to parent edge would be in two simple cycles).

9 Solutions

We have 3 very distinct types of solutions:

1. Storing parent information along the DFS path/stack.
2. Storing no extra information and instead finding it when we need to backtrack.
3. Storing child information along the DFS path/stack.

Some of these end up being slightly worse than others, but all can net a high score on the problem. We also believe studying them is useful, since the techniques in them can be useful for other problems.

9.1 Storing parent information

9.1.1 Storing the parent's index

Since each node sees at most 2 DFS-like nodes (one of which is its parent), we can simply record which of the 2 is its actual parent. Basically, for all DFS-like colors, we have 2 versions, specifying which node is the current node's parent.

The colors we need are: INIT, DONE, DFS_1/2, DFS_FRONT_1/2, DFS_NEW_1/2 (8 in total).

Each time we go to an INIT, we leave the current node in a DFS_FRONT color. Then its child can identify it as its parent and figure out the parent's index (0 or 1) among the DFS-like nodes it sees. Then the child is colored to a DFS_NEW color and we go to the parent. Now the parent returns to a DFS color and goes to the DFS_NEW color it sees (the child). After that, we proceed normally. Notice that this procedure allowed us to tell a node who its parent is and store that information in the node, while returning the parent to a regular color. Finally, when we need to backtrack, we just backtrack to the correct parent DFS-like node according to the index the current node stores. This solution needs $4N$ time, since the robot goes through each tree edge 4 times (and doesn't go through back-edges).

Code: `emil_cactus_par_8.cpp`

We can optimize this to 6 colors by removing the DFS_NEW one. We can merge it with the DFS_FRONT ones. Everything works normally, except that when we are in a DFS_FRONT node, we figure out whether it is the original DFS_FRONT or the original DFS_NEW based on whether it sees another DFS_FRONT node.

Code: `emil_cactus_par_6.cpp`

9.1.2 Marking the parent

The issue with the above ideas (besides needing $4N$ iterations) is that the number of DFS-like colors is multiplied by 2 due to storing the index of the parent in the node. A much better idea is to simply mark the parent on its own node. Then when we need to backtrack, if we have a back-edge, we just backtrack to the marked parent. The key is that we only need this marking if the current node has a back-edge. And, if that is the case, no node can have a back-edge to the current node's parent.

More concretely, we still follow a similar DFS/DFS_FRONT procedure. However, when we enter INIT we normally just backtrack, leaving it as INIT (which means we enter the DFS_FRONT parent, which is turned into DFS and we go back to the INIT child). This allows us to reuse DFS_FRONT as the parent marker too. However, if we detect that we need to mark the parent (when we enter an INIT node and we see 2 DFS-like nodes), we simply proceed without going back to the parent. Thus, the parent is naturally left in the DFS_FRONT state. This needs only 4 colors, but still $4N$ iterations.

Code: `emil_cactus_parimp_4.cpp`

9.2 Storing no extra information

In this solution we won't record any extra info while running the DFS. Instead, we'll figure out which node is our parent when we need to backtrack. If we only see one DFS-like node, we just go to it. If not, then we must have

a back-edge. Therefore, our parent cannot have a back-edge. Therefore, our parent would see only one DFS-like node (excluding the current node). On the other hand, any other DFS-like node would see at least 2 DFS-like nodes (its parent and its child); however, the root is an exception to this (so we can use an extra color for it – notice that this case does not matter in subtask 5).

The colors we need are: INIT, DONE, DFS, DFS_ROOT, PARSEEK, PARFOUND, DFS_PAR, DFS_NOTPAR (8 in total).

The color DFS_ROOT will be used only for the root. That way, if we see a DFS node and a DFS_ROOT node, we know the DFS one is our parent. So the only issue is if we see 2 DFS nodes. In that case, we mark the current node with PARSEEK, meaning we are looking for its parent. Then we go to a DFS node. Then (we know we are in this case, since we see a PARSEEK node), we check whether it is the parent or not (based on how many DFS-like nodes it sees). If yes, set it to DFS_PAR. Otherwise, set it to DFS_NOTPAR. In either case, go back to the PARSEEK node. There, we may need to do this again until we see a DFS_PAR node.

Once we see a DFS_PAR, we need to clean up the DFS_NOTPAR ones (if any), color the current node in PARFOUND and go to the DFS_NOTPAR. There, color it in DFS and go back to the PARFOUND node. Finally, set the current node to DONE and go to the DFS_PAR node. Notice that it will be overwritten with DFS (if exploring further), DONE (if backtracking) (PARSEEK cannot happen).

This solution does at most 6 steps per node with back-edge. In a cactus, at most half the nodes have back-edges (the maximum number of edges is $3N/2$). Therefore, in total we get $2N + 6N/2 = 5N$ iterations. Notice that it doesn't use the ordering of the adjacency list being consistent.

Code: `emil_cactus_find_8.cpp`

There are multiple possible optimizations, which can mostly be done in any order. We'll present one order of doing them, but you can also find codes for some other orders in the sample solutions.

9.2.1 Removing PARFOUND

First, we can remove PARFOUND, its only purpose is to tell the DFS_NOTPAR node that it needs to clear itself. However, this is the only situation in which we are ever in a DFS_NOTPAR node. Therefore, we can merge PARFOUND with PARSEEK.

Code: `emil_cactus_find_7_no-found.cpp`

9.2.2 Removing DFS_ROOT

We can also remove the DFS_ROOT and merge it with DFS with one key observation. Let's see what the issue is if we do this. Everything works, until we have a back-edge to the root. If we first try our actual parent, everything is fine. However, if we try the root, it looks just as if it is our parent (only sees one DFS-like node), so we'll mistakenly go to it. However, we can imagine this as something like a DFS (which keeps a stack of nodes), but at this point flipping the stack, i.e. swapping which side we're expanding from. The important invariant that all the DFS-like nodes form a single path still holds. Therefore, we'll still visit all nodes and then terminate (though this termination might just not happen from the root, which is not an issue). Notice that the new subtree of the old root cannot have back-edges to above the root (in the flipped stack). Additionally, the other end of the stack (the new root) has visited all of its other children, so it will never be a back-edge. Therefore this stack-flipping/rerooting operation will only happen once.

Code: `emil_cactus_find_6_no-found-root.cpp`

9.2.3 Removing DFS_NOTPAR

Next, we can also exploit the fact that we have consistent ordering in order to remove DFS_NOTPAR and merge it with DFS. This optimization will also lower the number of iterations to $3N$. When we try a node (i.e. when we see PARSEEK), we'll just either mark it with DFS_PAR or leave it as DFS and return to the PARSEEK node. Then when we are in PARSEEK (note that we are not actually in PARSEEK before we try the first of the 2 DFS nodes), we can see that either we found the parent (so we just backtrack to it) or did not (in which case we only see 2 DFS nodes). In that latter case we can just backtrack to the other DFS node (i.e. at the start always try the left DFS node and in this case always backtrack to the right DFS node). Besides removing one color, this also makes us only do 2 extra steps per back-edge, for a total of $3N$ steps, which also lets us get points from subtask 7.

Code: `emil_cactus_find_5_no-root-found-notpar.cpp`

9.2.4 Removing DFS_PAR

Finally, we can merge PARSEEK and DFS_PAR in a similar way to how we merged DFS_FRONT and DFS_NEW in the other type of solution. Now, when the current node is in this new PARSEEK_OR_PAR color, we treat it as PARSEEK when we see 2 DFS nodes or see another PARSEEK_OR_PAR node (which we treat as DFS_PAR). We also treat it as PARSEEK, if we are in a DFS node and see a PARSEEK_OR_PAR (it is just PARSEEK). However, if we are in PARSEEK_OR_PAR, but see no other PARSEEK_OR_PAR and see at most 1 DFS node, we must be in DFS_PAR.

Code: `emil_cactus_find_4_no-found-root-notpar-par.cpp`

9.2.5 Variations

Next are some interesting variations not covered above, that would be interesting to understand and show useful techniques for such problems. Additionally, it is interesting to see what the best solution is if the adjacency list is unordered (e.g. if the grader shuffles it each time).

9.2.6 Clearing DFS_NOTPAR with PARSEEK_OR_PAR

Let's see whether we can remove the DFS_PAR color (merge it with PARSEEK) without removing the DFS_NOTPAR color (and without depending on the consistent order of the adjacent nodes).

There is an issue when we try to clear DFS_NOTPAR and see two PARSEEK_OR_PAR nodes (can happen when there is a triangle, i.e. a cycle with 3 nodes). In this situation we can simply guess which node is the PARSEEK one and go to it (clearing to DFS, as usual). Then when we are in a PARSEEK_OR_PAR node, but see INITs, this must have happened and we must have guessed wrongly, so we just go to the adjacent PARSEEK_OR_PAR (while keeping that node as PARSEEK_OR_PAR) – this fixes the wrong guess with one extra step. Notice that in case the DFS_PAR had already visited all its other children, we may mistakenly think it is the real PARSEEK. However, this is not an issue, since this just means we backtrack the other way around the triangle but everything is still correct after that. This ends up with $5.5N$ iterations in total.

Code: `emil_cactus_find_5_no-found-root-par.cpp`

9.2.7 Lazy DFS_NOTPAR clearing

In all of the above solutions that do have DFS_NOTPAR we can actually just not clear it. Let's look at doing this on our simplest solution. When we are in PARSEEK and see a DFS_NOTPAR, we will just backtrack to the DFS node. This would leave some DFS_NOTPAR nodes lying around, but no node would have them as a back-edge. Additionally, that node's child cannot have any back-edge. Therefore, we never see both DFS and DFS_NOTPAR while traversing normally. So we just act normally or backtrack to the DFS_NOTPAR, if it is the only DFS-like node we see. This solution only does $3N$ steps, which means we can pass the last subtask even without any of the other optimizations.

Code: `emil_cactus_find_7_no-found_lazy-notpar.cpp`

The above argumentation only works until we remove the DFS_ROOT color. This is because once we switch the root the stack is flipped. There are no back-edges from the new subtree of the old root until we backtrack above the root. Then it is possible for some node to have a back-edge to a DFS_NOTPAR node. However, in that case it would also have a regular DFS node and that must be its parent. Additionally, such a node cannot have another back-edge. Therefore, we can just ignore the DFS_NOTPAR nodes unless we see no DFS node, in which case we backtrack to the DFS_NOTPAR node. All of this also works if we merge PARSEEK and DFS_PAR, so this is actually a cleaner resolution to the issue from before. Thus we get a 5 color, $3N$ iterations solution to the unordered version of the problem.

Code: `emil_cactus_find_5_no-found-root-par_lazy-notpar.cpp`

9.2.8 Lazy PARSEEK clearing

In the previous solutions, we merged PARSEEK and DFS_PAR. Note that DFS_PAR is needed only for clearing PARSEEK. Another idea is to simply not clear PARSEEK immediately (if we happen to guess the right parent, but just leave it there).

Of course, to have any chance, we need to only go to PARSEEK, if it looks like we might be answering its query, i.e. if we see 2 or more DFS-like nodes. There is still an issue with this – if we later mistakenly think we are answering the PARSEEK; this can only happen if we actually are in the middle of answering another PARSEEK and we are not the parent (if we just backtracked to the current node, we would only see 1 DFS node – we cannot have a back-edge since our child did). When this happens, we might go to the wrong PARSEEK and then think we are answering it that we are the wrong parent and go to its other adjacent DFS-like node. After that, we turn out to be in the middle of the DFS path with no PARSEEK around us, so we end up losing. This actually cannot happen on subtask 5, so it is a valid solution for it.

However, we can see that the issue comes from the current node's ex-children being marked as PARSEEK – they will always be earlier in the adjacency list than the new PARSEEK. Therefore, we can fix this solution by always answering the latest/rightmost PARSEEK in our adjacency list. This needs only 4 colors and does $3N$ iterations, so it is also a full solution.

Code: `emil_cactus_find_4_no-found-root-notpar-par_lazy-parseek.cpp`

9.3 Storing child information

One other idea, similar to the one storing the parent's index, is to store the child's index. Namely, whenever we go down towards a child, we can see what its index (among the DFS-like nodes) will be and store it in the current node. Then when we need to backtrack from a node and it sees multiple DFS-like nodes, we can guess which one the parent is. If we choose right, we are done. If we choose wrong, we can use the stored child indices to descend down to the correct parent (without changing any colors along the way). This sounds like it might need $O(N^2)$ steps, but we can notice that we go around each cycle at most once, this this would actually be just $3N$ steps.

Now we just need to figure out how to know that we are in this descend mode. For non-root nodes without back-edges, we can tell by simply checking whether the number of seen DFS-like nodes is 2 or 1: if 2, we are descending; if 1, we are in the regular mode. For the root, for now, we can just have a special color (and we never backtrack to it unless it is the only option). Finally, we can notice that we never need to descend through nodes with back-edges (since it is impossible to have a back-edge to a parent/grandparent of theirs), so they don't need to store child information. We can just have a special color for such nodes (found by seeing 2 DFS-like nodes when in an INIT node). Lastly, we need to not backtrack to such nodes, if we see regular DFS-like nodes. This solution uses 6 colors and needs $3N$ iterations.

Code: `emil_cactus_child_6.cpp`

We can optimize this solution in a similar way to our main one and simply remove the color for the root. This means that we may backtrack to it (if we guess wrongly). Then the root starts exploring its other subtrees (and we never encounter back-edges to the first subtree). Finally, we start backtracking through the first subtree (and exploring any unexplored branches). In this last part, the notations of child indices of the not-yet-backtracked nodes are wrong. However, we can never see back-edges to them, since that would break the cactus property (as that whole path is part of a cycle). Therefore, this is fine and would work. This uses 5 colors (and still $3N$ iterations). It is the best solution we have for the unordered version of the problem.

Code: `emil_cactus_child_5_no-root.cpp`

10 Even better solutions?

We do not know whether a 3 color solution exists or whether a 4 color one exists for the unordered version of the problem. However, we have proven (using a brute force) that a 2 color solution does not exist even for a path (even in the ordered version).

Similarly, we do not know whether a faster than $3N$ solution exists with a constant number of colors for the ordered or the unordered version of the problem.

Desired Solution

We begin by reformulating the problem. Let $pre_i = \sum_{j=1}^i d_j$. For each position $1 \leq i \leq n$, define $S(i)$ as the number of stars Ian can collect if he starts at position i . Then: $S(i) = |\{j \geq i \mid t_j \leq pre_j - pre_{i-1}\}|$

There are two neat approaches to solving this problem efficiently, both based on computing all values of $S(i)$.

Full Solution 1

We modify the condition, note that $t_j \leq pre_j - pre_{i-1} \iff pre_{i-1} \leq pre_j - t_j$. We define $a_j = pre_j - t_j$, then the formula for $S(i)$ becomes $S(i) = |\{j \geq i \mid a_j \geq pre_{i-1}\}|$. We compute $S(i)$ by enumerating i from right to left. We maintain a multiset that stores all current values of a_j for $j \geq i$. For each i , we first insert a_i , then we compute how many elements in the multiset are $\geq pre_{i-1}$.

This yields an $O(N \log N)$ solution using either a Binary Indexed Tree (Fenwick Tree) or an ordered set (e.g.,

`__gnu_pbds::tree` in C++). Time Complexity: $O(N \log N)$

Full Solution 2

Using the same simplification $S(i) = |\{j \geq i \mid a_j \geq pre_{i-1}\}|$, we now observe that if there exists $j < i$ such that $pre_{j-1} \leq pre_{i-1}$, then starting from i is never optimal. This means that we only need to consider starting positions i such that pre_{i-1} is the minimum in the prefix, we call these valid starting positions.

We compute $S(i)$ for these valid starting positions by sorting all a_j in non-increasing order. Then we iterate over all valid starting positions i in increasing order. We maintain a set A of indices $j \geq i$ such that $a_j \geq pre_{i-1}$. We remove any indices $j < i$ in A , since they are no longer valid. After the cleanup, $S(i) = |A|$. We can use a priority queue or a multiset to maintain and update A dynamically. Time Complexity: $O(N \log N)$

Subtask 1 ($N \leq 2000$)

These subtasks are intended for brute-force solutions. For each starting point i , simulate and count the amount of stars Ian can collect. Time Complexity: $O(N^2)$

Subtask 2 (at most 20 negative d_i 's)

This subtask supports Solution 2. Since at most 20 prefix sums may decrease (due to negative values), the number of valid starting positions is at most 20. By identifying all valid starting positions and simulate each in $O(N)$, we obtain a solution to pass this subtask. Time Complexity: $O(N|\{i : P_i < 0\}|)$

Subtask 3 (no positive d_i)

Since all $d_i \leq 0$, pre_i is a non-increasing sequence. We can use binary search to count the number of indices which satisfy the inequality $a_j \geq pre_{i-1}$ as described in the full solutions. Time Complexity $O(N \log N)$

Subtask 4

This subtask is intended to encourage a simplified version of Solution 1. Since the number of stars collected is at most 20, we can calculate the amount of elements in the multiset that are $\geq pre_{i-1}$ by maintain the top 20 largest elements in a fixed-size array, using `std::set` and `prev()` function, or use a `priority_queue` and pop elements greater than pre_{i-1} until none remain. Time Complexity: $O(N \log N + N \times \text{ans})$ or $O(N \log N \times \text{ans})$ depending on which data structure is used